

TP PROGRAMACIÓN IV · CÁTEDRA 2026

Roadmap

Sistema de Cine

Plan de ejecución ordenado, fase por fase, del catálogo público a la administración integral.

Documento vivo. Se actualiza sesión a sesión a medida que se completan fases o cambian decisiones. La fuente de verdad del análisis funcional (requisitos, casos de uso, modelo de datos) es externa a este documento; acá se traduce ese análisis en un plan de ejecución ordenado.

ESTRUCTURA DE CARPETAS

```
src/app/
  core/           # SupabaseService, guards, interceptors, modelos TS compartidos
  shared/         # UI primitives propias (botón, card, badge, spinner, modal...)
  layout/         # header, footer, shell con outlet
  features/
    catalogo/     # M02 - listado, detalle, búsqueda/filtro, reseñas
    salas-butacas/ # M03/M04 - mapa de butacas, selección, disponibilidad
    compra/       # M05/M06/M07 - carrito, candy/combos, cupones, pago mock (NgModule)
    fidelizacion/ # M08 - puntos, crédito, canjes
    entradas/     # M09 - ticket PDF + QR (vista cliente)
    cancelaciones/ # M10
    proximamente/ # M11 - estrenos, alertas
    perfil/       # M01 - registro, login, datos del cliente, Mis películas
    empleado/     # validación QR/código manual
    administracion/ # M12/M13 + ABM de todo lo anterior (NgModule)
  app.routes.ts   # composición de rutas lazy por feature
```

ROADMAP POR FASES

- ☒ Hecho
- ☒ En progreso
- ☐ Pendiente

Fase 0 — Fundaciones del proyecto

☒ Hecho

#	SUB-TAREA	ESTADO
0.1	Limpieza de repo y documentación base (README + CHANGELOG)	☒ Hecho
0.2	Estructura de carpetas (<code>features/*</code> + rutas lazy placeholder)	☒ Hecho
0.3	Supabase: cliente, <code>environment.ts</code> , <code>SupabaseService</code>	☒ Hecho
0.4	Design system base: variables SCSS + componentes <code>Button</code> / <code>Card</code>	☒ Hecho
0.5	Shell de layout: header, footer, fondo temático	☒ Hecho
0.6	Esquema SQL inicial en Supabase (tablas base + RLS)	☒ Hecho
0.7	Seed de datos de demo	☒ Hecho

Verificación 0.2 (hecha): `ng build` genera un chunk lazy por feature, `ng serve` levanta la app y navega entre las 10 páginas placeholder vía la navegación provisoria. `core/` / `shared/` / `layout/` se crean recién en 0.3/0.4/0.5, cuando tengan contenido real (git no versiona carpetas vacías).

Fase 1 — Catálogo público (M02)

 En progreso

Listado de películas (poster, nombre, duración, clasificación), detalle (sinopsis, géneros, reseñas con estrellas y promedio, funciones disponibles), buscador + filtro múltiple por género, destacado "3 más vendidas" en home.

#	SUB-TAREA	ESTADO
1.1	Capa de datos: modelos + <code>PeliculasService</code> (listado/detalle/destacadas) + RPC agregada de "más vendidas"	 Hecho
1.2	Página de listado (home): grilla de películas + destacado "3 más vendidas"	 Hecho
1.3	Buscador + filtro múltiple por género	 Hecho
1.4	Página de detalle de película (sinopsis, géneros, funciones disponibles, reseñas + promedio)	 Pendiente

Fase 2 — Salas, butacas y funciones (M03/M04)

 En progreso

Mapa visual de butacas (normal/accesible/VIP con diferenciación de color clara), disponibilidad en tiempo real, selección de función desde el detalle.

#	SUB-TAREA	ESTADO
2.1	Capa de datos: modelos (<code>Sala</code> , <code>Butaca</code> , <code>Funcion</code> , <code>ReservaButaca</code>) + <code>FuncionesService</code> (funciones disponibles por película)	 Hecho
2.2	Selección de función desde el detalle de película (fecha, horario, sala, tipo de proyección 2D/3D/4D/5D, idioma)	 Pendiente
2.3	Mapa visual de butacas: layout por sala (filas/columnas) con diferenciación de color normal/accesible/VIP	 Pendiente
2.4	Bloqueo temporal de butacas en selección: RPC transaccional sobre <code>reservas_butaca</code> + expiración por <code>expira_at</code>	 Pendiente
2.5	Disponibilidad en tiempo real (Supabase Realtime) reflejada en el mapa mientras otro usuario selecciona	 Pendiente
2.6	Confirmación de butacas seleccionadas → entrega el estado al carrito de la Fase 4	 Pendiente

Fase 3 — Autenticación y perfil (M01)

☐ Pendiente

Registro (mail, nombre, apellido, fecha nacimiento, tipo de sangre, color de ojos, vacaciones anuales), login/logout, perfil (datos, puntos, crédito, historiales) — sin romper la navegación de invitado.

#	SUB-TAREA	ESTADO
3.1	Formulario de registro (<code>raw_user_meta_data</code> : nombre, apellido, fecha de nacimiento, tipo de sangre, color de ojos, vacaciones anuales) + <code>AuthService</code>	☐ Pendiente
3.2	Login/logout, persistencia de sesión y guard de rutas que requieren sesión	☐ Pendiente
3.3	Página de perfil: ver/editar datos propios (sin poder tocar <code>rol</code> /saldos, protegido también por trigger en backend)	☐ Pendiente
3.4	Perfil: saldo de puntos y crédito + historial de movimientos (<code>movimientos_puntos</code> / <code>movimientos_credito</code>)	☐ Pendiente
3.5	Perfil: historial de compras propias (<code>ventas</code>) y accesos rápidos ("Mis películas")	☐ Pendiente
3.6	Navegación mixta invitado/registrado: header refleja sesión sin romper el flujo de compra anónima existente	☐ Pendiente

Fase 4 — Compra: entradas + Candy/combos + cupones (M05/M06/M07)

☐ Pendiente

Carrito (entradas + productos + combos), cupón, compra anónima vs registrada, validación de edad (RN04), pantalla de pago simulada, persistencia de venta.

#	SUB-TAREA	ESTADO
4.1	Estructura <code>NgModule</code> de la feature <code>compra</code> + <code>CarritoService</code> (signals: entradas seleccionadas, productos, combos)	☐ Pendiente
4.2	Candy Bar: listado de productos/combos por categoría, cantidad, agregar/quitar del carrito	☐ Pendiente
4.3	Aplicación de cupón: validación de tipo (<code>primera_compra</code> / <code>edad</code> / <code>general</code>), vigencia y porcentaje	☐ Pendiente
4.4	Checkout: datos de contacto, compra anónima vs registrada, validación de edad (RN04, <code>fecha_nacimiento_comprador</code>)	☐ Pendiente
4.5	Checkout registrado: uso opcional de crédito/puntos disponibles como medio de pago parcial	☐ Pendiente
4.6	Pantalla de pago simulada + RPC transaccional de confirmación (valida butacas, crea <code>ventas</code> / <code>venta_items</code> / <code>pagos</code> , acredita puntos)	☐ Pendiente
4.7	Pantalla de confirmación de compra (resumen, entradas emitidas)	☐ Pendiente

Fase 5 — Entradas y QR (M09, vista cliente)

☐ Pendiente

PDF de entrada + QR, pantalla "Mis entradas".

#	SUB-TAREA	ESTADO
5.1	Componente de visualización de QR por entrada (<code>entradas.codigo_qr</code>)	☐ Pendiente
5.2	Generación de PDF de entrada (función, sala, butaca, QR)	☐ Pendiente
5.3	Pantalla "Mis entradas": listado propio, separación próximas/pasadas, estado (<code>emitida</code> / <code>validada</code> / <code>cancelada</code>)	☐ Pendiente

Fase 6 — Panel de empleado: validación (M09, operación)

☐ Pendiente

Escaneo/ingreso manual de código, validación de entrada y retiro de Candy, invalidación de QR usado.

#	SUB-TAREA	ESTADO
6.1	Guard de rol <code>empleado</code> + layout mínimo del panel	☐ Pendiente
6.2	Escaneo de QR por cámara + ingreso manual de código como alternativa	☐ Pendiente
6.3	RPC de validación de entrada: marca <code>validada</code> , registra en <code>usos_qr</code> , rechaza código inexistente/ya usado	☐ Pendiente
6.4	Retiro de Candy: marca <code>ventas.candy_entregado_at</code> , independiente de la validación de entradas	☐ Pendiente
6.5	Historial de validaciones de la sesión del empleado en curso	☐ Pendiente

Fase 7 — Panel de administración: ABM base (M03/M04/M06/M07 admin)

☐ Pendiente

CRUD de películas, salas/butacas, funciones (asignación automática de sala + RN01/RN02/RN03), productos/categorías, combos, cupones y preventa.

#	SUB-TAREA	ESTADO
7.1	Estructura <code>NgModule</code> de la feature <code>administracion</code> + guard de rol <code>admin</code> + layout del panel	☐ Pendiente
7.2	ABM de películas y géneros (incluye <code>preventa_habilitada</code> / <code>precio_preventa</code>)	☐ Pendiente
7.3	ABM de salas y butacas (layout de filas/columnas, tipo normal/accesible/VIP)	☐ Pendiente
7.4	ABM de funciones: asignación de sala, validación de solapamiento (RN01/RN02/RN03, margen de 30 min ya reforzado en Postgres)	☐ Pendiente
7.5	ABM de productos/categorías y combos (con <code>combo_items</code>)	☐ Pendiente
7.6	ABM de cupones (tipo, vigencia, porcentaje)	☐ Pendiente

Fase 8 — Fidelización y crédito (M08)

☐ Pendiente

Acreditación de puntos, configuración y canje de recompensas, saldo e historial, no transferencia.

#	SUB-TAREA	ESTADO
8.1	Acreditación de puntos al confirmar una compra (integra con el RPC de la Fase 4, vía <code>movimientos_puntos</code>)	☐ Pendiente
8.2	ABM de recompensas desde administración (<code>recompensas</code> : tipo entrada/producto, costo en puntos)	☐ Pendiente
8.3	Canje de recompensas desde el perfil del cliente (<code>canjes</code> + débito de puntos)	☐ Pendiente
8.4	Vista de saldo e historial unificado de puntos y crédito en el perfil	☐ Pendiente
8.5	Verificación de la regla "no transferencia": puntos/crédito solo se mueven por operaciones propias del usuario vía RPC, nunca por edición directa	☐ Pendiente

Fase 9 — Cancelaciones (M10)

☐ Pendiente

Cancelación hasta 2 horas antes, liberación de butacas, generación de crédito.

#	SUB-TAREA	ESTADO
9.1	Cancelación de compra desde "Mis entradas"/perfil, con validación de ventana (hasta 2 hs antes de la función)	☐ Pendiente
9.2	RPC de cancelación: marca <code>ventas</code> / <code>venta_items</code> cancelados (libera el índice único de butaca), genera <code>movimientos_credito</code> , actualiza <code>peliculas.entradas_vendidas</code>	☐ Pendiente
9.3	Cancelación desde el panel de administración (con motivo, sin restricción de ventana horaria)	☐ Pendiente

Fase 10 — Próximamente y notificaciones (M11)

☐ Pendiente

Sección "Próximamente", alertas de estreno, preventa 7 días antes, notificación in-app al habilitarse la venta.

#	SUB-TAREA	ESTADO
10.1	Sección pública "Próximamente" (películas con preventa habilitada o estreno futuro)	☐ Pendiente
10.2	Suscripción a alertas de estreno desde el perfil (<code>alertas_estreno</code>)	☐ Pendiente
10.3	Preventa: habilitación de venta 7 días antes del estreno con <code>precio_preventa</code>	☐ Pendiente
10.4	Notificaciones in-app al habilitarse la preventa/venta (<code>notificaciones</code>) + indicador de no leídas	☐ Pendiente

Fase 11 — Reportes y estadísticas (M12)

☐ Pendiente

Facturación diaria, entradas vendidas, exportación PDF/Excel, gráficos de películas más vistas y producto más vendido.

#	SUB-TAREA	ESTADO
11.1	Reporte de facturación por día/rango de fechas	☐ Pendiente
11.2	Reporte de entradas vendidas por película/función/período	☐ Pendiente
11.3	Gráficos: películas más vistas y producto más vendido	☐ Pendiente
11.4	Exportación de reportes a PDF/Excel	☐ Pendiente

Fase 12 — Auditoría (M13)

☐ Pendiente

Registro y consulta de logs de acciones administrativas y de validación.

#	SUB-TAREA	ESTADO
12.1	Registro en <code>logs_actividad</code> desde las acciones administrativas de las Fases 7-11 (RPC/triggers)	☐ Pendiente
12.2	Registro en <code>logs_actividad</code> de las validaciones de QR/Candy del panel de empleado (Fase 6)	☐ Pendiente
12.3	Pantalla de consulta de logs en administración, con filtros por usuario/entidad/acción/fecha	☐ Pendiente

Fase 13 — PWA y pulido final

☐ Pendiente

Manifest + service worker, instalabilidad, revisión de RLS, accesibilidad, responsive final, despliegue a Vercel, README de arquitectura (RNF-008).

#	SUB-TAREA	ESTADO
13.1	Manifest + service worker + instalabilidad (Angular PWA)	☐ Pendiente
13.2	Revisión final de políticas RLS: cobertura de las 26 tablas y de cada rol (anon/authenticated/empleado/admin)	☐ Pendiente
13.3	Accesibilidad (a11y): contraste, foco, roles ARIA en componentes propios del design system	☐ Pendiente
13.4	Responsive final en todas las features	☐ Pendiente
13.5	Despliegue a Vercel + variables de entorno de producción	☐ Pendiente
13.6	README de arquitectura final (RNF-008)	☐ Pendiente

Convención transversal - Cada PR agrega una entrada nueva en `CHANGELOG.md`. - El `README.md` se actualiza cuando una fase introduce una decisión de arquitectura nueva. - Cada PR describe: Objetivo inicial / Qué se terminó haciendo / Resumen de cambios. Sin líneas de atribución al agente.

CONVENCIÓN TRANSVERSAL

- Cada PR agrega una entrada nueva en `CHANGELOG.md`.
- El `README.md` se actualiza cuando una fase introduce una decisión de arquitectura nueva.
- Cada PR describe: Objetivo inicial / Qué se terminó haciendo / Resumen de cambios. Sin líneas de atribución al agente.